شرح الواجهة ٢ — شاشة «اختيار الصف»

مشروع رِزمة · Flutter · شرح سطر بسطر مع سبب كل قرار

هذا الملف يشرح الواجهة الثانية من تصميم Figma (الإطار `اختيار الصف 02`): شريط علوي فيه زر رجوع وعنوان «اختر صفّك»، حقل «ما اسمك؟»، ثلاث مجموعات صفوف (1–4 و5–8 و9–12)، ملاحظة، وزر «متابعة».

هذه أول شاشة تفاعلية في التطبيق: فيها حقل نص وصف مختار وزر يتفعّل ويتعطّل. لذلك نستخدم هنا `StatefulWidget` لأول مرة، ونشرح لماذا.

تعتمد الشاشة على مكوّنات شُرحت في الملف الأول وهي: `PrimaryButton` و`AppColors` و`AppTextStyles` و`AppAssets`. لن نعيد شرحها هنا.

تذكير بالاتجاه: التطبيق RTL، فأول عنصر في `Row` يظهر على اليمين. في الشريط العلوي رتّبنا العناصر: زر الرجوع (يمين) ← العنوان الشخصية (يسار)، وهو نفس مظهر Figma.

ما أضفناه عن التصميم: لم يحدّد Figma حالة «لم يُختَر شيء بعد»، فأضفنا نصًّا إرشاديًّا «اكتب اسمك» وجعلنا الزر معطّلًا (باللون الرمادي `disabled` الموجود في التصميم) حتى يُكتب اسم ويُختار صف. أما الحالة المعروضة في Figma (اسم «أنس» وصف 5 مختار) فهي ما ستراه بعد إدخالها.

lib/core/models/student_profile.dart — نموذج بيانات الطالب

lib/core/models/student_profile.dart

يحمل الاسم والصف من شاشة الاختيار إلى الشاشة الرئيسية. نموذج صغير لا يحتاج حزمة (Freezed ونحوها) في هذه المرحلة.

الأسطر 1–2

```
/// بيانات الطالب التي تُجمع في شاشة اختيار الصف وتُعرض في الرئيسية.
class StudentProfile {
```

ماذا يفعل؟ تعليق وصنف `StudentProfile`.

لماذا بهذه الطريقة؟ صنف عادي بلا اعتماد على Flutter؛ يمكن اختباره كـ Dart صرف.

السطر 3

```
  const StudentProfile({required this.name, required this.grade});
```

ماذا يفعل؟ باني `const` بمعاملين مسمّيين مطلوبين: `name` و`grade`.

لماذا بهذه الطريقة؟ المعاملات المسمّاة تجعل الاستدعاء مقروءًا (`StudentProfile(name: ..., grade: 5)`) و`const` يتيح إنشاءه وقت الترجمة.

الأسطر 5–6

```
  final String name;
  final int grade;
```

ماذا يفعل؟ حقلان نهائيان: الاسم (نص) والصف (رقم 1–12).

لماذا بهذه الطريقة؟ `final` لأن الملف الشخصي غير قابل للتعديل بعد إنشائه؛ نصنع واحدًا جديدًا عند أي تغيير.

الأسطر 8–9

```
8     /// مثال: الصف الخامس
9     String get gradeLabel => 'الصف ${_ordinals[grade - 1]}';
```

ماذا يفعل؟ خاصية `gradeLabel` تعيد «الصف الخامس» مثلًا.

لماذا بهذه الطريقة؟ الشاشة الرئيسية تعرض الصف بالأحرف («الصف الخامس») بينما نخزّن الرقم (5). وضعنا التحويل في النموذج بدل الشاشة ليُعاد استخدامه ويُختبر مرة واحدة. `grade - 1` لأن الفهارس تبدأ من الصفر.

الأسطر 11–24

```
11    static const _ordinals = [
12      'الأول',
13      'الثاني',
14      'الثالث',
15      'الرابع',
16      'الخامس',
17      'السادس',
18      'السابع',
19      'الثامن',
20      'التاسع',
21      'العاشر',
22      'الحادي عشر',
23      'الثاني عشر',
24    ];
```

ماذا يفعل؟ قائمة الأعداد الترتيبية العربية من «الأول» إلى «الثاني عشر».

لماذا بهذه الطريقة؟ قائمة صريحة أسلم من محاولة توليد العدد الترتيبي عربيًا برمجيًا، فالمؤنّث/المذكّر وصيغة «الحادي عشر» و«الثاني عشر» لها قواعدها. `static const` لأنها لا تتغيّر ولا تُنسخ مع كل كائن.

السطر 25

```
25  }
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/grade_selection/grade_selection_screen.dart — شاشة اختيار الصف

lib/features/grade_selection/grade_selection_screen.dart

الأسطر 1–2

```
1   import 'package:flutter/material.dart';
2   import 'package:flutter_svg/flutter_svg.dart';
```

ماذا يفعل؟ استيراد Material وSVG.

لماذا بهذه الطريقة؟ الأيقونات SVG.

الأسطر 4–9

```
import '../../core/constants/app_assets.dart';
import '../../core/models/student_profile.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/primary_button.dart';
import '../home/home_screen.dart';
```

ماذا يفعل؟ استيراد الأصول والنموذج والألوان والأنماط والزر المشترك وشاشة الرئيسية.

لماذا بهذه الطريقة؟ نستورد الرئيسية لأن الزر ينتقل إليها بعد الحفظ.

الأسطر 11–21

```
class _GradeGroup {
  const _GradeGroup({
    required this.title,
    required this.hint,
    required this.grades,
  });

  final String title;
  final String hint;
  final List<int> grades;
}
```

ماذا يفعل؟ صنف خاص `_GradeGroup` يصف مجموعة صفوف: العنوان (الصفوف 1–4) والتلميح (صور وصوت أكثر) وقائمة الأرقام.

لماذا بهذه الطريقة؟ بدل كتابة المجموعات الثلاث يدويًا ثلاث مرات في الواجهة، نحوّلها إلى بيانات ثم نولّد الواجهة منها بحلقة. إضافة مجموعة رابعة لاحقًا تصير سطرًا واحدًا.

الأسطر 23–35

```
const _groups = [
  _GradeGroup(title: 'الصفوف 1-4', hint: 'صور وصوت أكثر', grades: [1, 2, 3, 4]),
  _GradeGroup(
    title: 'الصفوف 5-8',
    hint: 'تدريبات وتقدّم',
    grades: [5, 6, 7, 8],
  ),
  _GradeGroup(
    title: 'الصفوف 9-12',
    hint: 'ملخّصات وامتحانات',
    grades: [9, 10, 11, 12],
  ),
];
```

ماذا يفعل؟ الثابت `_groups` بالمجموعات الثلاث: 1–4 و5–8 و9–12 مع عناوينها وتلميحاتها.

لماذا بهذه الطريقة؟ النصوص مطابقة لتصميم Figma حرفيًا. أرقام كل مجموعة مرتّبة تصاعديًا (1،2،3،4) لأن RTL يضع الأول على اليمين فيظهر «1» يمينًا كما في Figma.

الأسطر 37–39

```
/// الشاشة 02 – اختيار الصف
class GradeSelectionScreen extends StatefulWidget {
  const GradeSelectionScreen({super.key});
```

ماذا يفعل؟ تعليق وصنف `GradeSelectionScreen` من نوع `StatefulWidget`.

لماذا بهذه الطريقة؟ نحتاج حالة: الصف المختار ونص الاسم. لذا `StatefulWidget` وليس `StatelessWidget`.

الأسطر 41–43

```
  @override
  State<GradeSelectionScreen> createState() => _GradeSelectionScreenState();
}
```

ماذا يفعل؟ `createState` تنشئ كائن الحالة.

لماذا بهذه الطريقة؟ نمط ثابت في Flutter يربط الودجت بحالته.

الأسطر 45–47

```
class _GradeSelectionScreenState extends State<GradeSelectionScreen> {
  final _nameController = TextEditingController();
  int? _selectedGrade;
```

ماذا يفعل؟ صنف الحالة وفيه: `_nameController` للتحكم بنص الاسم و`_selectedGrade` للصف المختار (قابل لأن يكون `null`).

لماذا بهذه الطريقة؟ `TextEditingController` يعطينا قراءة النص الحالي والاستماع إلى تغييره. `int?` لأن لا صف مختار في البداية.

الأسطر 49–53

```
  @override
  void initState() {
    super.initState();
    _nameController.addListener(() => setState(() {}));
  }
```

ماذا يفعل؟ `initState` : نضيف مستمعًا يستدعي `setState` كلما تغيّر نص الاسم.

لماذا بهذه الطريقة؟ زر المتابعة يتفعّل ويتعطل بحسب الاسم، فيجب إعادة بناء الشاشة مع كل حرف. `setState(() {})` الفارغة تكفي لأن القيمة تُقرأ من الـ controller.

الأسطر 55–59

```
  @override
  void dispose() {
    _nameController.dispose();
    super.dispose();
  }
```

ماذا يفعل؟ `dispose` : نتخلّص من الـ controller.

لماذا بهذه الطريقة؟ التخلّص من المتحكمات ضروري لتجنّب تسريب الذاكرة؛ `super.dispose()` أخيرًا حسب التوصية.

الأسطر 61–62

```
61    bool get _canContinue =>
62        _selectedGrade != null && _nameController.text.trim().isNotEmpty;
```

ماذا يفعل؟ خاصية `_canContinue` تعيد `true` إن اختير صف وكُتب اسم غير فارغ.

لماذا بهذه الطريقة؟ `trim()` يتجاهل المسافات (فلا يُقبل اسم من مسافات). نجمع الشرط في خاصية واحدة ليُقرأ بسهولة.

الأسطر 64–68

```
64    void _continue() {
65      final profile = StudentProfile(
66        name: _nameController.text.trim(),
67        grade: _selectedGrade!,
68      );
```

ماذا يفعل؟ الدالة `_continue` تنشئ `StudentProfile` من الاسم المقصوص والصف.

لماذا بهذه الطريقة؟ `_selectedGrade!` آمنة لأن الزر لا يعمل إلا إذا `_canContinue` صحيحة (أي أن الصف غير فارغ).

الأسطر 69–72

```
69      Navigator.of(context).pushAndRemoveUntil(
70        MaterialPageRoute<void>(builder: (_) => HomeScreen(profile: profile)),
71        (route) => false,
72      );
```

ماذا يفعل؟ ينتقل إلى `HomeScreen` ويمسح كل الشاشات السابقة (`pushAndRemoveUntil` مع `(route) => false`).

لماذا بهذه الطريقة؟ بعد إكمال التعريف لا نريد أن يرجع المستخدم إلى الترحيب بزر الرجوع؛ الرئيسية هي نقطة البداية الجديدة.

السطر 73

```
73    }
```

ماذا يفعل؟ إغلاق الدالة.

لماذا بهذه الطريقة؟ نهاية `_continue`.

الأسطر 75–77

```
75    @override
76    Widget build(BuildContext context) {
77      return Scaffold(
```

ماذا يفعل؟ دالة `build` وتبدأ بـ `Scaffold`.

لماذا بهذه الطريقة؟ الهيكل الأساسي.

السطر 78

```
78        body: SafeArea(
```

ماذا يفعل؟ `SafeArea` تحمي الشاشة من شريط الحالة والنتوءات.

لماذا بهذه الطريقة؟ الخلفية فاتحة والنص داكن فلا يلزم تغيير أيقونات شريط الحالة هنا.

الأسطر 79–82

```
79          child: SingleChildScrollView(
80            child: Column(
81              crossAxisAlignment: CrossAxisAlignment.stretch,
82              children: [
```

ماذا يفعل؟ `SingleChildScrollView` ثم `Column` ممتد، وأول عناصره الشريط العلوي.

لماذا بهذه الطريقة؟ التمرير مهم هنا: عند ظهور لوحة المفاتيح تضيق المساحة ويجب أن نستطيع الوصول إلى الحقول.

السطر 83

```
83                _TopBar(onBack: () => Navigator.of(context).maybePop()),
```

ماذا يفعل؟ الشريط العلوي `_TopBar` مع تمرير دالة الرجوع `maybePop`.

لماذا بهذه الطريقة؟ `maybePop` ترجع فقط إن وُجدت شاشة سابقة ولا تُسبّب خطأً إن لم توجد.

الأسطر 84–85

```
84                Padding(
85                  padding: const EdgeInsets.fromLTRB(20, 12, 20, 24),
```

ماذا يفعل؟ حشوة 20 يمين/يسار و12 أعلى و24 أسفل.

لماذا بهذه الطريقة؟ قيم Figma.

الأسطر 86–89

```
86                  child: Column(
87                    crossAxisAlignment: CrossAxisAlignment.stretch,
88                    spacing: 20,
89                    children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 20 بين الأقسام.

لماذا بهذه الطريقة؟ `spacing` تغني عن الفواصل اليدوية.

السطر 90

```
90                    _NameField(controller: _nameController),
```

ماذا يفعل؟ حقل الاسم `_NameField` ويمرّر المتحكم.

لماذا بهذه الطريقة؟ نمرّر الـ controller الذي تملكه الحالة، فنستطيع قراءة النص من هنا.

الأسطر 91–97

```
                    for (final group in _groups)
                      _GradeGroupSection(
                        group: group,
                        selected: _selectedGrade,
                        onSelected: (grade) =>
                            setState(() => _selectedGrade = grade),
                      ),
```

ماذا يفعل؟ حلقة تولّد قسمًا لكل مجموعة، وتمرّر الصف المختار ودالة الاختيار.

لماذا بهذه الطريقة؟ عند الاختيار نستدعي `setState` فتُعاد الرسم وتُظلَّل البلاطة المختارة. لاحظ أن الحالة تعيش في الشاشة ثم تنزل للأبناء كمعاملات (رفع الحالة للأعلى).

الأسطر 98–101

```
                    Text(
                      'يمكنك تغيير الصف لاحقًا من حسابك.',
                      style: AppTextStyles.caption13,
                    ),
```

ماذا يفعل؟ الملاحظة «يمكنك تغيير الصف لاحقًا من حسابك.»

لماذا بهذه الطريقة؟ نص تطميني وفق التصميم بنمط `caption13`.

الأسطر 102–108

```
                    PrimaryButton(
                      label: _selectedGrade == null
                          ? 'متابعة'
                          : 'متابعة · الصف $_selectedGrade',
                      iconAsset: AppAssets.fwdGrade,
                      onPressed: _canContinue ? _continue : null,
                    ),
```

ماذا يفعل؟ زر المتابعة: النص «متابعة» قبل الاختيار و«متابعة · الصف N» بعده، وأيقونة السهم، والإجراء `_continue` أو `null`.

لماذا بهذه الطريقة؟ تمرير `null` يعطّل الزر تلقائيًا (رمادي). النص الديناميكي هو ما يعرضه Figma («متابعة · الصف 5»).

الأسطر 109–118

```
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة والصنف.

الأسطر 120–123

```
120 class _TopBar extends StatelessWidget {
121   const _TopBar({required this.onBack});
122
123   final VoidCallback onBack;
```

ماذا يفعل؟ ودجت `_TopBar` بلا حالة ويستقبل `onBack`.

لماذا بهذه الطريقة؟ ودجت منفصل ليبقى `build` الرئيسي قصيرًا ومقروءًا.

الأسطر 125–128

```
125   @override
126   Widget build(BuildContext context) {
127     return Padding(
128       padding: const EdgeInsets.fromLTRB(20, 16, 20, 4),
```

ماذا يفعل؟ `build` وحشوة الشريط: 20 يمين/يسار و16 أعلى و4 أسفل.

لماذا بهذه الطريقة؟ قيم Figma.

الأسطر 129–131

```
129       child: Row(
130         spacing: 12,
131         children: [
```

ماذا يفعل؟ `Row` بمسافة 12 بين العناصر.

لماذا بهذه الطريقة؟ أول عنصر على اليمين: زر الرجوع.

الأسطر 132–135

```
132           Semantics(
133             button: true,
134             label: 'رجوع',
135             excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` تعرّف الزر لقارئ الشاشة باسم «رجوع».

لماذا بهذه الطريقة؟ الزر أيقونة بلا نص، فبدون هذا الوصف لن يعرف مستخدم قارئ الشاشة وظيفته.

الأسطر 136–139

```
136             child: Material(
137               color: AppColors.mute,
138               borderRadius: BorderRadius.circular(14),
139               clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بلون `mute` وزوايا 14 وقصّ.

لماذا بهذه الطريقة؟ نفس سبب `InkWell` : `PrimaryButton` يحتاج `Material` ليرسم التموّج.

الأسطر 140–141

```
140              child: InkWell(
141                onTap: onBack,
```

ماذا يفعل؟ `InkWell` عند الضغط ينفّذ `onBack`.

لماذا بهذه الطريقة؟ الإجراء يأتي من الخارج.

الأسطر 142–152

```
142                child: SizedBox(
143                  width: 44,
144                  height: 44,
145                  child: Center(
146                    child: SvgPicture.asset(
147                      AppAssets.back,
148                      width: 22,
149                      height: 22,
150                    ),
151                  ),
152                ),
```

ماذا يفعل؟ مربع بحجم 44×44 فيه أيقونة السهم بحجم 22 في المنتصف.

لماذا بهذه الطريقة؟ 44 هو الحد الأدنى المريح للمس تقريبًا (توصية 44–48). أيقونة «رجوع» في Figma مرسومة بسهم لليمين = اتجاه الرجوع في RTL، فلا نقلبها.

الأسطر 153–155

```
153              ),
154            ),
155          ),
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الزر.

الأسطر 156–164

```
156          Expanded(
157            child: Column(
158              crossAxisAlignment: CrossAxisAlignment.start,
159              children: [
160                Text('خطوة 1 من 2', style: AppTextStyles.caption13),
161                Text('اختر صفّك', style: AppTextStyles.heading22),
162              ],
163            ),
164          ),
```

ماذا يفعل؟ `Expanded` فيه عمود: «خطوة 1 من 2» ثم «اختر صفّك».

لماذا بهذه الطريقة؟ `Expanded` يملأ المساحة بين الزر والشخصية. `start` = اليمين في RTL فيبدأ النص من جهة زر الرجوع.

السطر 165

```
165           SvgPicture.asset(AppAssets.mascot56, width: 56, height: 56),
```

ماذا يفعل؟ الشخصية بحجم 56 في أقصى اليسار.

لماذا بهذه الطريقة؟ آخر عنصر في الصف = أقصى اليسار.

الأسطر 166–170

```
166         ],
167       ),
168     );
169   }
170 }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 172–175

```
172 class _NameField extends StatelessWidget {
173   const _NameField({required this.controller});
174
175   final TextEditingController controller;
```

ماذا يفعل؟ ودجت `_NameField` يستقبل الـ controller.

لماذا بهذه الطريقة؟ ودجت مستقل لتغليف تفاصيل مظهر الحقل.

الأسطر 177–178

```
177   @override
178   Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` .

لماذا بهذه الطريقة؟ يبدأ بتعريف الحدود.

الأسطر 179–182

```
179     const border = OutlineInputBorder(
180       borderRadius: BorderRadius.all(Radius.circular(16)),
181       borderSide: BorderSide(color: AppColors.line, width: 2),
182     );
```

ماذا يفعل؟ ثابت محلي `border` : مستطيل بزوايا 16 وحدّ بسُمك 2 بلون `line` .

لماذا بهذه الطريقة؟ نعرّفه مرة ونعيد استخدامه في الحالات الثلاث (عادي ومفعّل وفي التركيز) بدل تكرار نفس الإعدادات.

الأسطر 184–187

```
184     return Column(
185       crossAxisAlignment: CrossAxisAlignment.stretch,
186       spacing: 6,
187       children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 6.

لماذا بهذه الطريقة؟ بين العنوان والحقل.

السطر 188

```
188         Text('ما اسمك؟', style: AppTextStyles.body16Strong),
```

ماذا يفعل؟ عنوان الحقل «ما اسمك؟».

لماذا بهذه الطريقة؟ نص مستقل فوق الحقل (نمط `body16Strong`).

الأسطر 189–190

```
189         SizedBox(
190           height: 52,
```

ماذا يفعل؟ `SizedBox` بارتفاع 52 يحيط بالحقل.

لماذا بهذه الطريقة؟ يثبّت الارتفاع كما في Figma بدل ارتفاع الحقل الافتراضي.

الأسطر 191–196

```
191           child: TextField(
192             controller: controller,
193             style: AppTextStyles.input17,
194             textInputAction: TextInputAction.done,
195             textCapitalization: TextCapitalization.words,
196             cursorColor: AppColors.teal,
```

ماذا يفعل؟ `TextField` بالمتحكم ونمط `input17` وإجراء لوحة المفاتيح «تم» وكتابة الأحرف الأولى كبيرة ولون المؤشّر teal.

لماذا بهذه الطريقة؟ العربية بلا أحرف كبيرة لكن الخاصية تفيد إن كتب المستخدم بالإنجليزية. اتجاه النص يُستنتج تلقائيًا من اللغة (RTL) فيبدأ الكتابة من اليمين.

الأسطر 197–201

```
197             decoration: InputDecoration(
198               hintText: 'اكتب اسمك',
199               hintStyle: AppTextStyles.input17.copyWith(
200                 color: AppColors.disabled,
201               ),
```

ماذا يفعل؟ زخرفة الحقل: نص إرشادي «اكتب اسمك» باللون الرمادي.

لماذا بهذه الطريقة؟ إضافة غير موجودة في Figma ولكنها لازمة لحقل فارغ.

الأسطر 202–204

```
            filled: true,
            fillColor: AppColors.card,
            contentPadding: const EdgeInsets.symmetric(horizontal: 16),
```

ماذا يفعل؟ خلفية بيضاء مملوءة وحشوة أفقية 16.

لماذا بهذه الطريقة؟ نفس مظهر Figma (أبيض بحدّ رمادي).

الأسطر 205–206

```
            border: border,
            enabledBorder: border,
```

ماذا يفعل؟ نفس الحدّ للحالة العادية والمفعّلة.

لماذا بهذه الطريقة؟ يلغي حدود Material الافتراضية.

الأسطر 207–209

```
            focusedBorder: border.copyWith(
              borderSide: const BorderSide(color: AppColors.teal, width: 2),
            ),
```

ماذا يفعل؟ عند التركيز يتحوّل الحد إلى teal.

لماذا بهذه الطريقة؟ تغذية بصرية تبيّن أن الحقل نشط. `copyWith` تحافظ على نصف القطر وتغيّر اللون فقط.

الأسطر 210–216

```
            ),
          ),
        ),
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 218–223

```
class _GradeGroupSection extends StatelessWidget {
  const _GradeGroupSection({
    required this.group,
    required this.selected,
    required this.onSelected,
  });
```

ماذا يفعل؟ ودجت `_GradeGroupSection` يستقبل المجموعة والصف المختار ودالة الاختيار.

لماذا بهذه الطريقة؟ قسم واحد (عنوان + 4 بلاطات) يتكرّر 3 مرات.

الأسطر 225–227

```
  final _GradeGroup group;
  final int? selected;
  final ValueChanged<int> onSelected;
```

ماذا يفعل؟ الحقول: المجموعة والمختار (`int?`) و `ValueChanged<int>` .

لماذا بهذه الطريقة؟ `ValueChanged<int>` دالة تستقبل رقم الصف المختار.

الأسطر 229–234

```
  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      spacing: 8,
      children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 8.

لماذا بهذه الطريقة؟ بين العنوان وصف البلاطات.

الأسطر 235–243

```
        Row(
          crossAxisAlignment: CrossAxisAlignment.baseline,
          textBaseline: TextBaseline.alphabetic,
          spacing: 8,
          children: [
            Text(group.title, style: AppTextStyles.section18),
            Text(group.hint, style: AppTextStyles.caption13),
          ],
        ),
```

ماذا يفعل؟ صف العنوان: «الصفوف 1–4» ثم التلميح «صور وصوت أكثر» بمحاذاة خط الأساس.

لماذا بهذه الطريقة؟ `CrossAxisAlignment.baseline` يضع قاعدة الخطين على سطر واحد رغم اختلاف حجميهما (18 و13)، وهذا ما يفعله Figma. يتطلب تحديد `textBaseline` .

الأسطر 244–256

```
        Row(
          spacing: 10,
          children: [
            for (final grade in group.grades)
              Expanded(
                child: _GradeTile(
                  grade: grade,
                  selected: grade == selected,
                  onTap: () => onSelected(grade),
                ),
              ),
          ],
        ),
```

ماذا يفعل؟ صف البلاطات: لكل رقم `Expanded` يحوي `_GradeTile` بمسافة 10 بينها.

لماذا بهذه الطريقة؟ `Expanded` يوزّع العرض بالتساوي (كل بلاطة نحو 80 على شاشة 390) فتتكيّف الشاشة مع كل الأعراض بدل عرض ثابت. الضغط يستدعي `onSelected(grade)`.

الأسطر 257–260

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 262–267

```
class _GradeTile extends StatelessWidget {
  const _GradeTile({
    required this.grade,
    required this.selected,
    required this.onTap,
  });
```

ماذا يفعل؟ ودجت `_GradeTile` يستقبل الرقم وهل هو مختار ودالة الضغط.

لماذا بهذه الطريقة؟ بلاطة واحدة.

الأسطر 269–271

```
  final int grade;
  final bool selected;
  final VoidCallback onTap;
```

ماذا يفعل؟ الحقول الثلاثة.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 273–275

```
273   @override
274   Widget build(BuildContext context) {
275     final borderColor = selected ? AppColors.teal : AppColors.line;
```

ماذا يفعل؟ `build` ويحسب لون الحد: teal إن اختيرت، `line` إن لا.

لماذا بهذه الطريقة؟ نحسبه مرة ونستعمله لاحقًا.

الأسطر 277–281

```
277     return Semantics(
278       button: true,
279       selected: selected,
280       label: 'الصف $grade',
281       excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` يعلن أنها زر ومختار أو لا، ويقرأ «الصف N».

لماذا بهذه الطريقة؟ مستخدم قارئ الشاشة يسمع «الصف 5، مختار».

الأسطر 282–284

```
282       child: AnimatedContainer(
283         duration: const Duration(milliseconds: 150),
284         height: 64,
```

ماذا يفعل؟ `AnimatedContainer` بمدة 150 مللي ثانية وارتفاع 64.

لماذا بهذه الطريقة؟ تغيّر اللون عند الاختيار يصير انتقالًا ناعمًا بدل قفزة. الارتفاع 64 من Figma.

الأسطر 285–289

```
285         decoration: BoxDecoration(
286           color: selected ? AppColors.teal : AppColors.card,
287           borderRadius: BorderRadius.circular(18),
288           border: Border.all(color: borderColor, width: 2),
289         ),
```

ماذا يفعل؟ خلفية teal إن اختيرت وإلا أبيض، وزوايا 18، وحدّ بسُمك 2.

لماذا بهذه الطريقة؟ بلاطة مختارة: أخضر مصمت. غير مختارة: أبيض بحدّ رمادي. مطابق للتصميم.

الأسطر 290–294

```
290         child: Material(
291           type: MaterialType.transparency,
292           child: InkWell(
293             borderRadius: BorderRadius.circular(16),
294             onTap: onTap,
```

ماذا يفعل؟ `Material` شفاف و `InkWell` بزوايا 16 يلتقط الضغط.

لماذا بهذه الطريقة؟ الخلفية الملونة مرسومة في `AnimatedContainer`، فنضع `Material` شفافًا فوقها كي يظهر التموّج. نصف قطر 16 أقل بـ2 من 18 لأن الحد بسُمك 2.

الأسطر 295–303

```
            child: Center(
              child: Text(
                '$grade',
                style: AppTextStyles.gradeNumber24.copyWith(
                  color: selected ? AppColors.card : AppColors.ink,
                ),
              ),
            ),
          ),
```

ماذا يفعل؟ الرقم في المنتصف بنمط `gradeNumber24`، أبيض عند الاختيار وداكن عند عدمه.

لماذا بهذه الطريقة؟ تباين عالٍ في الحالتين.

الأسطر 304–308

```
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.